

FA7 Part 2

Diche, Jasmine & Hermosada, Joshua

10/07/2026

Frequency Distribution and Empirical Rule Analysis

```
# 1. Input Data
lunch_money <- c(150,120,200,100,280,80,250,200,100,150,
                 100,120,150,200,150,100,150,280,280,200,
                 200,80,200,120,280,150,150,250,120,150,
                 200,250,250,150,120,250,80,280,120,250,
                 100,120,120,120,100,80,280,120,100,100)

# 2. Frequency Distribution Table
freq_table <- as.data.frame(table(lunch_money))
colnames(freq_table) <- c("Lunch_Money", "Frequency")
freq_table$Lunch_Money <- as.numeric(as.character(freq_table$Lunch_Money))
freq_table$Relative_Freq_Pct <- (freq_table$Frequency / length(lunch_money)) * 100
freq_table$Cumulative_Freq <- cumsum(freq_table$Frequency)

cat("=== FREQUENCY DISTRIBUTION TABLE ===\n")

## === FREQUENCY DISTRIBUTION TABLE ===
print(freq_table)

##   Lunch_Money Frequency Relative_Freq_Pct Cumulative_Freq
## 1          80         4             8             4
## 2         100         8            16            12
## 3         120        10            20            22
## 4         150         9            18            31
## 5         200         7            14            38
## 6         250         6            12            44
## 7         280         6            12            50

# 3. Summary Statistics
mu <- mean(lunch_money)
sd_sample <- sd(lunch_money)
N <- length(lunch_money)
sd_pop <- sd_sample * sqrt((N - 1) / N)

cat("\n=== SUMMARY STATISTICS ===\n")

##
## === SUMMARY STATISTICS ===
cat(sprintf("Mean (mu): %.2f\n", mu))

## Mean (mu): 165.00
```

```

cat(sprintf("Sample Standard Deviation (s): %.2f\n", sd_sample))

## Sample Standard Deviation (s): 66.43
cat(sprintf("Population Standard Deviation (sigma): %.2f\n", sd_pop))

## Population Standard Deviation (sigma): 65.76
# 4. Empirical Rule Percentages
pct_1sd <- mean(lunch_money >= (mu - sd_pop) & lunch_money <= (mu + sd_pop)) * 100
pct_2sd <- mean(lunch_money >= (mu - 2*sd_pop) & lunch_money <= (mu + 2*sd_pop)) * 100
pct_3sd <- mean(lunch_money >= (mu - 3*sd_pop) & lunch_money <= (mu + 3*sd_pop)) * 100

cat("\n=== EMPIRICAL RULE PERCENTAGES ===\n")

##
## === EMPIRICAL RULE PERCENTAGES ===
cat(sprintf("Data within 1 SD (%.2f to %.2f): %.2f%%\n", mu - sd_pop, mu + sd_pop, pct_1sd))

## Data within 1 SD (99.24 to 230.76): 68.00%
cat(sprintf("Data within 2 SD (%.2f to %.2f): %.2f%%\n", mu - 2*sd_pop, mu + 2*sd_pop, pct_2sd))

## Data within 2 SD (33.47 to 296.53): 100.00%
cat(sprintf("Data within 3 SD (%.2f to %.2f): %.2f%%\n", mu - 3*sd_pop, mu + 3*sd_pop, pct_3sd))

## Data within 3 SD (-32.29 to 362.29): 100.00%
# 5. Define Plotting Function BEFORE Calling It
x_seq <- seq(0, 350, length = 300)
y_norm <- dnorm(x_seq, mean = mu, sd = sd_pop)

create_sd_plot <- function(k, col_line, label_text, actual_pct) {
  hist(lunch_money,
    probability = TRUE,
    col = "aliceblue",
    border = "gray70",
    main = paste0("Range: ", label_text),
    sub = sprintf("Coverage: %.2f%%", actual_pct),
    xlab = "Lunch Money (PHP)",
    ylab = "Density",
    breaks = seq(60, 300, by = 20),
    ylim = c(0, max(y_norm) * 1.15))

  lines(x_seq, y_norm, col = "black", lwd = 2)
  abline(v = mu, col = "red", lwd = 2, lty = 1)

  lower_bound <- mu - k * sd_pop
  upper_bound <- mu + k * sd_pop
  abline(v = c(lower_bound, upper_bound), col = col_line, lwd = 2.5, lty = 2)

  x_shade <- seq(lower_bound, upper_bound, length = 100)
  y_shade <- dnorm(x_shade, mean = mu, sd = sd_pop)
  polygon(c(lower_bound, x_shade, upper_bound), c(0, y_shade, 0),
    col = adjustcolor(col_line, alpha.f = 0.25), border = NA)

```

```

legend("topright",
      legend = c("Normal Curve", "Mean", label_text),
      col = c("black", "red", col_line),
      lty = c(1, 1, 2),
      lwd = 2,
      bg = "white",
      cex = 0.7)
}

```

Standard Deviation Plots

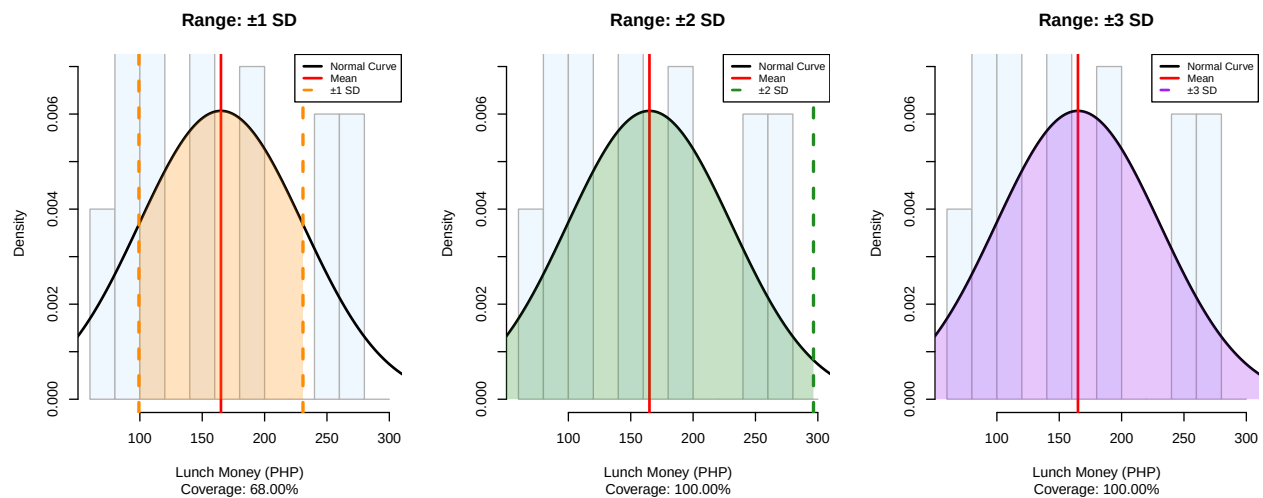
6. Render Plots Grid

```
par(mfrow = c(1, 3))
```

```

create_sd_plot(k = 1, col_line = "darkorange", label_text = "±1 SD", actual_pct = pct_1sd)
create_sd_plot(k = 2, col_line = "forestgreen", label_text = "±2 SD", actual_pct = pct_2sd)
create_sd_plot(k = 3, col_line = "purple", label_text = "±3 SD", actual_pct = pct_3sd)

```



```
par(mfrow = c(1, 1))
```